

Q1) Define classical logic gates and explain how they differ from quantum gates.

Definition of Classical Logic Gates:

Classical logic gates are the basic building blocks of **classical digital circuits**. They take **binary inputs (0 or 1)**, perform a **logical operation**, and produce a **binary output (0 or 1)** according to classical Boolean algebra.

Examples of classical logic gates include:

- **AND Gate:** Output is 1 only if both inputs are 1.
- **OR Gate:** Output is 1 if at least one input is 1.
- **NOT Gate:** Inverts the input ($0 \rightarrow 1, 1 \rightarrow 0$).
- **XOR, NAND, NOR, XNOR** are other commonly used gates

2. Difference Between Classical and Quantum Gates:		
Aspect	Classical Logic Gates	Quantum Logic Gates
Basic Unit of Information	Operate on bits that can only be 0 or 1.	Operate on qubits, which can be in a superposition of 0 and 1.
Type of Operation	Perform irreversible logical operations (e.g., AND, OR lose input information).	Perform reversible unitary operations that preserve information.
Mathematical Representation	Represented by Boolean functions (truth tables).	Represented by unitary matrices that act on quantum states.
Example Operation	NOT gate changes $0 \rightarrow 1$ and $1 \rightarrow 0$ deterministically.	Quantum X gate (analog of NOT) changes qubit states, but can also work on superpositions like α
Reversibility	Most gates (like AND, OR) are not reversible, because outputs don't uniquely determine input	All quantum gates are reversible — you can always get the input state back from the output.
Information Loss	Information can be lost in the process (e.g., two inputs $\rightarrow$ one output).	No information is lost; quantum gates preserve probabilities and coherence.
Parallelism	Process one possible input at a time.	Can process multiple inputs simultaneously due to superposition.
Examples	AND, OR, NOT, NAND, NOR, XOR, XNOR.	Pauli-X, Hadamard (H), CNOT, Toffoli, and Quantum Phase Gates.
Hardware	Implemented using transistors in classical circuits.	Implemented using quantum systems like photons, ions, or superconducting qubits.
Output Nature	Always gives a definite binary result.	Gives a probabilistic output after measurement.
Entanglement	Cannot create entanglement between bits.	Can create entanglement between qubits, which is key to quantum computing power.

3. Example Comparison:

- **Classical NOT Gate:**
 - Input: 0 → Output: 1
 - Input: 1 → Output: 0
- **Quantum X Gate (Quantum NOT):**
 - Input: $|0\rangle \rightarrow$ Output: $|1\rangle$
 - Input: $|1\rangle \rightarrow$ Output: $|0\rangle$
 - Input: $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \rightarrow$ Output: $\frac{1}{\sqrt{2}}(|1\rangle + |0\rangle)$ (superposition preserved)

Q2) Differentiate between the classical and quantum parts of Shor’s algorithm.

Aspect	Classical Part of Shor’s Algorithm	Quantum Part of Shor’s Algorithm
Purpose / Function	Performs arithmetic and logical computations before and after the quantum step, such as choosing numbers and verifying results.	Performs the period-finding step, which is the core quantum speedup of the algorithm.
Main Task	Converts the factoring problem into a period-finding problem using modular arithmetic.	Finds the period (r) of a function using quantum parallelism and the Quantum Fourier Transform (QFT).
Operations Used	Uses classical arithmetic, Euclidean algorithm, and modular exponentiation to prepare the problem.	Uses quantum gates, superposition, entanglement, and interference to find the function’s periodicity.
Input Type	Takes classical integers N (number to be factored) and a (random co-prime to N).	Uses quantum registers initialized in superposition to represent many possible inputs at once.
Mathematical Step	Computes $f(x) = a^x \bmod N$ classically to define the function.	Evaluates $f(x) = a^x \bmod N$ for all x simultaneously using quantum parallelism.
Core Technique	Based on number theory and classical logic.	Based on quantum mechanics, specifically superposition and Fourier interference.
↓		
Use of Superposition / Entanglement	Not used — operates on definite values.	Used extensively to explore many possibilities in parallel.
Transform Applied	None — purely deterministic computations.	Uses Quantum Fourier Transform (QFT) to extract the period of the modular function.
Reversibility	Classical steps are irreversible (information may be lost).	All operations are reversible (unitary quantum evolution).
Measurement	Performed only at the end to interpret results.	Measurement collapses quantum states after QFT to obtain information about the period.
Result	Uses the discovered period r to compute the prime factors of N using the relation $\gcd(a^{r/2} \pm 1, N)$.	Provides the period r efficiently — the quantum “speedup” step.
Time Complexity	Polynomial in $\log N$, but slower than the quantum step.	Exponential speedup — reduces the hardest step to polynomial time using quantum interference.
Example of Task	Choose random a , check if $\gcd(a, N) > 1$, compute factors once period is known.	Use qubits to find the smallest r such that $a^r \bmod N = 1$.

Q3) State and explain the Deutsch Algorithm. What problem does it solve?

The **Deutsch Algorithm** is one of the first and simplest examples of a **quantum algorithm**. It was proposed by **David Deutsch (1985)** to show how a **quantum computer** can solve a problem **faster than any classical computer** — even if only by a small amount.

It is a **two-qubit quantum algorithm** that demonstrates **quantum parallelism** — It serves as a **simple example** showing how **quantum superposition and interference** can be used to gain computational speedup.

The Deutsch Algorithm solves the following simple problem:

Given a function $f(x)$ that takes a single **bit input** (0 or 1) and returns a single **bit output** (0 or 1), determine whether $f(x)$ is **constant** or **balanced**.

- The function $f(x)$ can be one of four possible cases:
 1. $f(0) = 0, f(1) = 0 \rightarrow$ **Constant**
 2. $f(0) = 1, f(1) = 1 \rightarrow$ **Constant**
 3. $f(0) = 0, f(1) = 1 \rightarrow$ **Balanced**
 4. $f(0) = 1, f(1) = 0 \rightarrow$ **Balanced**

On a **classical computer**, to determine whether $f(x)$ is constant or balanced, you must evaluate **both** $f(0)$ **and** $f(1)$. That means you need **two function evaluations**. The **Deutsch Algorithm** shows that a **quantum computer can determine this with only one function evaluation** using quantum parallelism. This is achieved by applying the function to a **superposition** of inputs instead of testing them one by one.

Let's go through the Deutsch Algorithm step by step.

Start with two qubits:

$$|\psi_0\rangle = |0\rangle \otimes |1\rangle$$

Step 2: Apply Hadamard Gates

Apply Hadamard (H) gates to both qubits to create a superposition:

$$|\psi_1\rangle = (H \otimes H)|0, 1\rangle = \frac{1}{2}(|0\rangle + |1\rangle)(|0\rangle - |1\rangle)$$

Now the system is in a **superposition** of all possible input states (0 and 1).

Step 3: Apply the Oracle U_f

The quantum oracle encodes the function $f(x)$ and acts as:

$$U_f|x, y\rangle = |x, y \oplus f(x)\rangle$$

where $\oplus$ means XOR.

After applying U_f , the second qubit flips based on $f \downarrow$.

After applying U_f , the second qubit flips based on $f(x)$.

Resulting state:

$$|\psi_2\rangle = \frac{1}{2}[(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle](|0\rangle - |1\rangle)$$

Step 4: Apply Hadamard to the First Qubit Again

Apply another Hadamard gate to the **first qubit**:

$$H[(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle]$$

After simplification, we get:

$$|\psi_3\rangle = \begin{cases} |0\rangle & \text{if } f(0) = f(1) \text{ (constant)} \\ |1\rangle & \text{if } f(0) \neq f(1) \text{ (balanced)} \end{cases}$$

Step 5: Measurement

Measure the **first qubit**:

- If result = $|0\rangle$, function is **constant**.
- If result = $|1\rangle$, function is **balanced**.

Only **one measurement** (and one evaluation of $f(x)$) is required!

Quantum Circuit Diagram

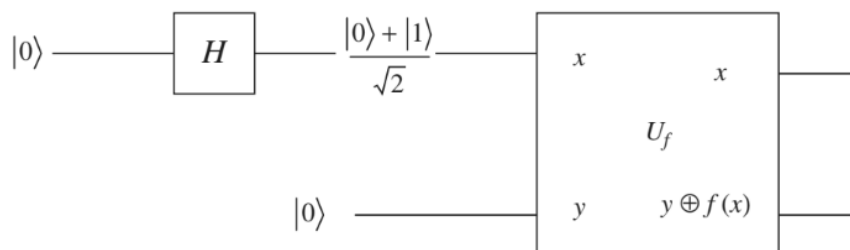


Figure 9.4 Circuit diagram for $U_f|x, y\rangle = |x, y \oplus f(x)\rangle$ where we take the first qubit to be in a superposition and the second qubit to be $|0\rangle$

Q4 What role does the Quantum Fourier Transform (QFT) play in Shor's algorithm?

Shor's algorithm is a quantum algorithm for **factoring large integers** efficiently — something classical computers can't do in practical time.

The key step in Shor's algorithm is finding the **period r** of a function:

$$f(x) = a^x \bmod N$$

where N is the number to be factored and a is an integer $< N$.

Once this **period r** is found, classical math is used to compute the **factors** of N .

✓ So, the hardest step — and the one giving quantum speed-up — is **finding the period r** .

That's where the **Quantum Fourier Transform (QFT)** comes in.

The **Quantum Fourier Transform (QFT)** is the quantum analogue of the classical discrete Fourier transform (DFT).

It converts a quantum state from the **computational basis** into the **frequency (phase) basis**.

Mathematically, for an n -qubit state $|x\rangle$,

$$QFT|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi i xy/2^n} |y\rangle$$

The QFT can be implemented efficiently on a quantum computer using **Hadamard gates** and **controlled phase rotations** in $O(n^2)$ time,

while the classical Fourier transform takes $O(2^n)$ time — an exponential difference.

1. Extracting the Hidden Period

Shor's algorithm relies on finding the period r of the modular exponentiation function $f(x) = a^x \bmod N$. ☞

- **Before QFT:** The quantum computer performs the modular exponentiation using **quantum parallelism**, which creates a large superposition where the period r is hidden in the **phases** of the state's amplitude (the calculation is done, but the answer is obscured).
- **Applying QFT:** The QFT acts like a frequency analyzer. It transforms the state from the input basis to the frequency basis. This transformation causes the probability amplitudes to interfere: ☞
 - The amplitudes corresponding to the **period (r) and its multiples** constructively interfere (add up).
 - The amplitudes corresponding to all other values destructively interfere (cancel out).

2. Enabling Efficient Measurement

Without the QFT, a measurement on the register would simply yield a random x value, and the period would still be unknown.

- **After QFT:** The interference amplifies the signal corresponding to the period. The state is transformed so that the probability of measuring a value related to r is extremely high. ⓘ
- **Measurement:** A final measurement of the output register yields a result k that is a multiple of R/r , where R is the size of the register. This value k is then used in a classical fraction-finding step (**Continued Fractions Algorithm**) to efficiently calculate the period r . ⓘ

1. Extracting Phase Information (The Exponential Speed-up)

Shor's algorithm uses quantum parallelism to encode all possible values of $a^x \pmod{N}$ into the phases of a quantum state. The state, just before the QFT is applied, is a superposition of states whose amplitudes contain information related to the period r . ⓘ

The QFT acts as a **super-efficient phase estimation tool**. Classically, calculating the phase of a superposition of L states takes time proportional to L^2 . The QFT calculates this phase information in time proportional to $L \log L$, which is an **exponential speed-up** for large inputs.

In short, the **QFT is the "engine" of Shor's algorithm**: it harnesses quantum interference to efficiently isolate the hidden periodicity information encoded in the quantum state

Q5 Describe how multi-controlled gates (e.g., Toffoli gate) can be used for quantum logic operations.

The role of **multi-controlled gates**, such as the **Toffoli gate** (or Controlled-Controlled-NOT, CCCN), is to execute logical operations conditional upon the state of *multiple* input qubits. They are crucial because they enable complex classical and quantum logic and are the key to achieving **universal quantum computation**.

1. Defining Multi-Controlled Gates

A multi-controlled gate is a unitary operation U on $n + m$ qubits:

$$U = C^n U'$$

where U' is a simple unitary gate (like a NOT or phase shift) that is applied to m target qubits **only if** all n control qubits are simultaneously in the $|1\rangle$ state.

The Toffoli gate is a 3-qubit quantum gate:

- Two control qubits
- One target qubit

It flips (applies NOT) on the target **only if both control qubits are |1⟩**.

Symbol: CCNOT

Action:

$$|a, b, c\rangle \rightarrow |a, b, c \oplus (a \cdot b)\rangle$$

Where:

- a, b = control qubits
- c = target qubit
- $\oplus$ = XOR operation

C1	C2	T	T'
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Role in Quantum Logic Operations

Multi-controlled gates are essential for the implementation of complex algorithms:

- **Universal Quantum Computation:** The Toffoli gate (a three-qubit gate) combined with any single-qubit gate (like the Hadamard or a phase gate) forms a **universal set of quantum gates**. This means that any quantum circuit or algorithm can be decomposed into a sequence of these basic, multi-controlled and single-qubit gates.
- **Arbitrary Control:** They allow the construction of arbitrary controlled operations needed for algorithms like **Shor's algorithm** and **Grover's algorithm**. For example, in quantum arithmetic, you need to apply a complex operation (like modular exponentiation) only if a whole register of control qubits is in a specific state. This requires multi-controlled operations beyond the simple CNOT.
- **Quantum Error Correction:** Toffoli gates are used in quantum error correction codes to detect and correct bit-flip and phase-flip errors. They perform conditional logic required for syndrome extraction and error diagnosis.
- **Entanglement Creation and Manipulation:** Multi-controlled gates help create or modify entanglement between qubits. By applying controlled operations, qubits can be correlated in specific ways required by quantum protocols.

Q6. Describe the Deutsch–Jozsa Algorithm and show how it provides exponential speedup over classical algorithms.

The **Deutsch–Jozsa algorithm** is an extension of the Deutsch algorithm. It was the first quantum algorithm to show an **exponential speedup** over its classical deterministic counterpart, solving a slightly more complex problem regarding a hidden function's property.

The Deutsch–Jozsa problem is to determine a global property of a hidden function f , where f takes an n -bit input and produces a 1-bit output,

$$f : \{0, 1\}^n \rightarrow \{0, 1\}$$

The function f is guaranteed to be one of two types: **Constant:** returns the same output (0 or 1) for all inputs, or **Balanced:** returns 0 for **half** of the inputs and 1 for the **other half**.

Classical Complexity

Classical Requirement

A classical computer, to *guarantee* a correct answer, must query the function f for up to $2^{n-1} + 1$ different inputs in the worst-case scenario:

- If the first 2^{n-1} queries all return the same result (e.g., 0), the computer still cannot be sure if the function is constant ($f(x) = 0$ for all x) or balanced (the very next query might return 1).
- The computer must query $2^{n-1} + 1$ inputs to achieve certainty. This scales **exponentially** with the size of the input n .

Time complexity: $O(2^n)$ evaluations (exponential growth)

Quantum Solution

Quantum Advantage

The Deutsch-Jozsa algorithm solves the problem with a **single query** to the quantum oracle U_f , regardless of the number of input qubits n .

- By leveraging **quantum parallelism**, the algorithm simultaneously evaluates $f(x)$ for all 2^n inputs.
- The **interference** step then aggregates this massive amount of information into the single final measurement, yielding the constant or balanced result with **100% certainty**.

Since the quantum complexity is $O(1)$ and the deterministic classical complexity is $O(2^n)$, the algorithm achieves an **exponential speedup**.

Time complexity: $O(1)$

Iska koi jarurat nahi hai bus ese hi hai

The Algorithm Steps

The algorithm operates on an $n + 1$ qubit register, where the first n qubits encode the input and the final qubit is the target.

1. **Initialization:** The $n + 1$ qubits are initialized to $|\psi_0\rangle = |0\rangle^{\otimes n} |1\rangle$.
2. **Initial Hadamard Gates:** Apply a **Hadamard gate (H)** to all $n + 1$ qubits. This creates a uniform superposition of all 2^n input states on the first n qubits:

$$|\psi_1\rangle = \left(\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \right) \otimes \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right)$$

3. **Quantum Oracle (U_f):** The hidden function f is applied once via a **quantum oracle** U_f . Due to **quantum parallelism** and the **phase kickback** trick, the oracle computes $f(x)$ for all 2^n inputs simultaneously and encodes the output into the phase of the input register:

$$|\psi_2\rangle = \left(\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \right) \otimes \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right)$$

4. **Final Hadamard Gates:** Apply a Hadamard gate to only the first n input qubits. This causes **interference** based on the global phase structure set by the oracle:

$$|\psi_3\rangle = \left(\frac{1}{2^n} \sum_{z \in \{0,1\}^n} \left(\sum_x (-1)^{f(x)} (-1)^{x \cdot z} \right) |z\rangle \right) \otimes \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right)$$

$$|\psi_3\rangle = \left(\frac{1}{2^n} \sum_{z \in \{0,1\}^n} \left(\sum_x (-1)^{f(x)} (-1)^{x \cdot z} \right) |z\rangle \right) \otimes \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right)$$

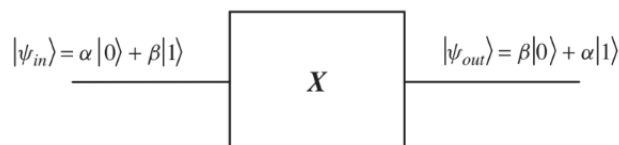
- **Crucially:** If f is **constant**, all phase terms interfere destructively except for the $|0\rangle^{\otimes n}$ term.
- If f is **balanced**, the phase terms for the $|0\rangle^{\otimes n}$ state interfere destructively, causing its amplitude to be zero.

5. **Measurement:** Measure the first n qubits.

- If the result is $|0\rangle^{\otimes n}$ (all zeros), the function is **constant**.
- If the result is **any other state** (any non-zero bit string), the function is **balanced**.

Q7 Describe the working of the Pauli-X, Pauli-Y, and Pauli-Z gates with their matrix representations.

The **Pauli gates** are **quantum analogs** of the classical NOT operation and are among the most basic and essential quantum logic gates. They are named after **Wolfgang Pauli**, who introduced the **Pauli matrices** in quantum mechanics to describe spin operators. Each Pauli gate acts on a **single qubit**, changing its **state on the Bloch sphere**.



3. Pauli-X Gate

Matrix Representation

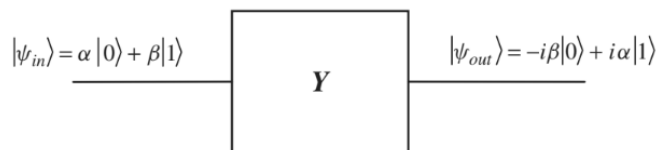
$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Effect on Basis States

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle$$

Action (Explanation)

- The Pauli-X gate acts like a **classical NOT gate** — it **flips** the qubit state.
- On the Bloch sphere, it performs a **rotation of 180°** around the X-axis.



4. Pauli-Y Gate

Matrix Representation

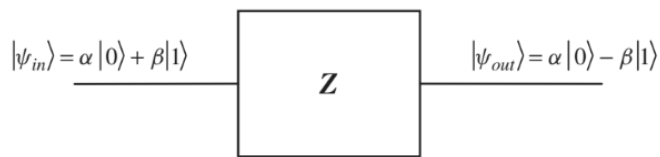
$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Effect on Basis States

$$Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle$$

Action (Explanation)

- The Pauli-Y gate performs a **bit-flip and a phase-flip simultaneously**.
- On the Bloch sphere, it corresponds to a **180° rotation around the Y-axis**.
- It introduces a **complex phase factor ($\pm i$)** during the flip.



5. Pauli-Z Gate

Matrix Representation

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Effect on Basis States

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle$$

Action (Explanation)

- The **Pauli-Z gate** leaves $|0\rangle$ unchanged and flips the **phase** of $|1\rangle$.
- On the **Bloch sphere**, it performs a **rotation of 180°** around the **Z-axis**.
- It **does not change the probability amplitudes** — only their **relative phase**.

Q8 What is Grover's Search Algorithm? Briefly describe its purpose and key advantage over classical search methods.

Grover's Search Algorithm is a **quantum algorithm** developed by **Lov Grover in 1996**.

It is designed to **search an unsorted database** or solve a search problem **quadratically faster** than any **classical algorithm**.

The primary purpose of Grover's algorithm is to **find a unique item**. Grover's algorithm finds a specific item among **N possibilities** in **only about $\sqrt{N}$ steps**, whereas a classical computer would need **N steps** on average.

You are given:

- A function $f(x)$ that outputs **1** only for the **desired input** (the "marked" item), and **0** for all other inputs.

Mathematically:

$$f(x) = \begin{cases} 1 & \text{if } x = x^*, \\ 0 & \text{otherwise.} \end{cases}$$

Key Advantage Over Classical Search

Grover's algorithm achieves a **quadratic speedup** over the best possible classical search methods, which is its most significant advantage.

Classical Complexity

For a classical computer searching an unsorted list of N items, the only way to guarantee finding the target item is to check, on average, $N/2$ items, and in the worst case, N items.

- **Classical Time Complexity:** $O(N)$ (linear time).

Quantum Complexity

Grover's algorithm only requires a number of queries proportional to the square root of the total number of items to find the target item with high probability.

- **Quantum Time Complexity:** $O(\sqrt{N})$ (quadratic speedup).

For a database of $N = 1,000,000$ items:

- **Classical Search** requires about **500,000** checks on average.
- **Grover's Algorithm** requires only about $\sqrt{1,000,000} = 1,000$ checks (iterations).

Yeh niche wala ese hi hai

Grover's algorithm works through a process of repeated amplitude amplification:

1. **Initial Superposition:** The algorithm starts by putting all N items into an equal superposition (using the **Hadamard gate**), meaning the search space is explored simultaneously (**quantum parallelism**).
2. **Grover Iteration:** The core of the algorithm is the repeated application of the **Grover Operator**, which consists of two main steps:
 - **The Oracle:** Identifies the marked item and flips its phase (amplitude is unchanged).
 - **Diffusion Operator:** **Inverts the amplitudes about the average.** This boosts the amplitude of the marked item while suppressing the amplitudes of all unmarked items.
3. **Measurement:** After the optimal number of iterations ($\approx \frac{\pi}{4} \sqrt{N}$), a measurement is performed. Because the marked item's amplitude has been amplified to nearly 1, the measurement is highly likely to yield the correct answer.

Q9 What problem does Deutsch's Algorithm solve, and how does it differ from a classical approach?

Q3...

Q10 Describe the Z-Y decomposition of a single-qubit rotation and provide its mathematical form.

The **Z-Y decomposition** of a single-qubit rotation is a fundamental identity in quantum computing that proves any arbitrary rotation of a single qubit can be achieved by combining rotations about only the \$Z\$ and \$Y\$ axes of the Bloch sphere.

This decomposition is crucial for understanding **universality**, as it shows that a limited set of gates is sufficient to perform any single-qubit operation.

Mathematical Form

An arbitrary single-qubit unitary operation $\mathbf{U}$ (which is an $SU(2)$ matrix) can be written as the product of three rotation gates:

$$\mathbf{U} = \mathbf{R}_Z(\gamma)\mathbf{R}_Y(\beta)\mathbf{R}_Z(\alpha)$$

Where:

- $\mathbf{R}_Z(\theta)$ is a rotation around the Z-axis by angle θ .
- $\mathbf{R}_Y(\theta)$ is a rotation around the Y-axis by angle θ .
- α, β , and γ are real angles that specify the rotation, with $0 \leq \alpha, \gamma < 2\pi$ and $0 \leq \beta \leq \pi$.

Matrix Representation

The full matrix form of the decomposition is:

$$\mathbf{U}(\alpha, \beta, \gamma) = \begin{pmatrix} e^{-i\gamma/2} & 0 \\ 0 & e^{i\gamma/2} \end{pmatrix} \begin{pmatrix} \cos(\beta/2) & -\sin(\beta/2) \\ \sin(\beta/2) & \cos(\beta/2) \end{pmatrix} \begin{pmatrix} e^{-i\alpha/2} & 0 \\ 0 & e^{i\alpha/2} \end{pmatrix}$$

Multiplying these matrices together gives:

$$\mathbf{U}(\alpha, \beta, \gamma) = \begin{pmatrix} e^{-i(\alpha+\gamma)/2} \cos(\beta/2) & -e^{i(\alpha-\gamma)/2} \sin(\beta/2) \\ e^{-i(\alpha-\gamma)/2} \sin(\beta/2) & e^{i(\alpha+\gamma)/2} \cos(\beta/2) \end{pmatrix}$$

Working Principle

Any rotation on the **Bloch Sphere** can be viewed as moving a vector from its initial position to its final position.

1. **First $R_Z(\alpha)$:** This initial rotation around the Z-axis positions the state vector at a specific longitude on the Bloch sphere, preparing it for the main rotation.
2. **$R_Y(\beta)$:** This rotation around the Y-axis moves the vector in terms of latitude, bringing it to the desired final position.
3. **Second $R_Z(\gamma)$:** This final rotation around the Z-axis fine-tunes the final phase or longitude without altering the achieved latitude from the R_Y rotation.

This means that any single quantum gate can be replaced by a sequence of R_Z and R_Y gates, which is essential for physically implementing quantum computers, as it simplifies the required hardware operations.

Q11 Explain the steps involved in Shor's Factorization Algorithm for integer factorization.

Shor's Algorithm, developed by **Peter Shor in 1994**, is a **quantum algorithm** that can factor large integers **exponentially faster** than the best known classical algorithms.

It shows the **true power of quantum computing**, since factoring large numbers is the mathematical foundation of **RSA encryption** — one of the most used encryption systems in the world.

$$f(x) = a^x \mod N$$

The algorithm is a hybrid process, alternating between classical (efficient) steps and a single, powerful quantum step.

1. Classical Reduction (The Setup)

A classical computer performs the initial setup and problem simplification:

1. **Input:** Start with the large composite number N to be factored.
2. **Trivial Checks:** Check if N is even or a power of a prime (e.g., $N = a^b$). If so, the factors are trivial and the algorithm stops.
3. **Choose Random Base:** Choose a random integer a such that $1 < a < N$.
4. **Find GCD:** Compute the **Greatest Common Divisor (GCD)** of a and N using the efficient classical **Euclidean algorithm**.
 - If $\text{GCD}(a, N) \neq 1$, then $\text{GCD}(a, N)$ is a factor of N , and the problem is solved.
 - If $\text{GCD}(a, N) = 1$, proceed to the quantum part.

The factoring problem has now been reduced to the problem of finding the **period r** (or **order**) of the function $f(x) = a^x \mod N$.

2. Quantum Period Finding (The Speedup)

The quantum computer uses the **Quantum Phase Estimation** procedure to find the period r of $f(x)$ with a single computation:

1. **Initial Superposition:** Initialize two quantum registers: an input register (for x) in a massive superposition of all possible x values, and an output register (for $f(x)$) initialized to $|0\rangle$.
2. **Modular Exponentiation:** Run the quantum circuit for **modular exponentiation** (U_f) on the registers. Due to **quantum parallelism**, this calculates $a^x \bmod N$ for *all* possible x values simultaneously.

$$|\Psi_1\rangle \propto \sum_x |x\rangle |a^x \bmod N\rangle$$

The period r is now encoded in the phases of the input register, but is still hidden.

3. **Quantum Fourier Transform (QFT):** Apply the **Quantum Fourier Transform (QFT)** to the input register. The QFT uses **quantum interference** to amplify the amplitudes of states related to the periodic structure (r) and suppress all others. This is the source of the exponential speedup.
4. **Measurement:** Measure the input register. The result k will be a value that is a multiple of R/r (where R is the size of the register) with high probability.

3. Classical Post-Processing (The Conclusion)

The classical computer takes the measurement result k and determines the final factors:

1. **Find Period r :** Use the measurement k and the size of the register R to approximate the fraction k/R . The classical **Continued Fractions Algorithm** is used to efficiently find the integer denominator of this fraction, which yields the period r .
2. **Factor N :** Use the found period r to calculate the factors p and q of N based on the following identity derived from number theory:

$$\text{GCD}(a^{r/2} \pm 1, N) \text{ yields a factor.}$$

This step involves a final application of the efficient classical Euclidean algorithm.

The successful non-trivial factor is then output, completing the factorization of N .

Q12 What are controlled gates? Explain the working of a CNOT gate with its truth table and matrix.

Controlled gates are multi-qubit quantum operations that apply a specific unitary transformation to one or more **target qubits** only when a set of **control qubits** are found to be in a particular state,

typically the $|1\rangle$ state. Examples of Controlled Gates as **CNOT** (Controlled-NOT) **CZ** (Controlled-Z) **Toffoli**(CCNOT) **CY** (Controlled-Y)

CNOT:

The **CNOT** gate (Controlled-NOT) is a **two-qubit gate** where:

- The **first qubit** acts as the **control**.
- The **second qubit** acts as the **target**.

It **flips (NOT operation)** the target qubit **only if the control qubit is $|1\rangle$** . If the control qubit is $|0\rangle$, then nothing happens to the target qubit. If the control qubit is $|1\rangle$, then the NOT or X matrix is applied to the target cubit.

✔ Truth Table of the CNOT Gate			
Control (Q_1)	Target (Q_2)	Output Control	Output Target
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

So, mathematically:

$$CNOT(|a, b\rangle) = |a, b \oplus a\rangle$$

where $\oplus$ is addition modulo 2 (XOR).

To write the matrix representation of the controlled NOT gate, we have to do it with respect to the states $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$. The matrix will be a 4×4 matrix. The matrix representation of this gate is given by

$$CN = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

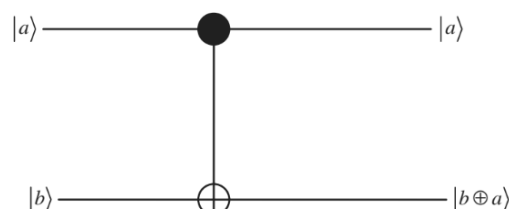


Figure 8.4 Circuit diagram representation of a controlled NOT gate

Q13 Explain how the reversible nature of quantum gates contrasts with classical logic gates.

The **reversible nature of quantum gates** is a fundamental difference that sharply contrasts with the operation of most standard **classical logic gates**. This contrast arises from the necessity in quantum mechanics to preserve information, a requirement tied to the mathematical nature of quantum operators.

1. Reversibility and Information Loss

Reversibility means that the original input can always be uniquely determined from the output of the gate.

- **Quantum Gates:** All fundamental quantum gates (U) are **unitary operations**. A unitary operation is always reversible because its inverse ($U^{-1} = U^\dagger$) is well-defined and also a unitary operation. 
 - **Implication:** When a quantum gate acts on a state, no information is lost, which is essential because quantum computation relies on maintaining the coherence and superposition of the qubits.
- **Classical Gates (Standard):** Most elementary classical gates, such as **AND**, **OR**, and **NAND**, are **irreversible**.
 - **Implication:** They map multiple possible input states to a single output state, destroying information about the original input. For example, if the output of an AND gate is '0', the input could have been 00, 01, or 10. You cannot reverse the operation to know the exact original input.

2. The Relationship to Energy and Entropy

This mathematical distinction has physical implications related to thermodynamics and information:

- **Information Loss (Classical):** Landauer's Principle states that the erasure of information (which happens in an irreversible gate) necessarily costs energy and generates heat (increases entropy). 
- **Information Preservation (Quantum):** Because quantum gates are reversible and preserve information, they theoretically do not dissipate heat or generate entropy during the computation itself (only during the final measurement). This is one reason quantum computers hold promise for ultra-low-power computation.

Q14 What problem does Shor's algorithm solve, and why is it significant in quantum computing?

Problem upper hai pad liyo...

Significance in Quantum Computing

Shor's algorithm holds profound significance, primarily because it delivers an **exponential speedup** over classical computation for a globally relevant problem, threatening modern cryptography.

1. Proof of Exponential Speedup

Shor's algorithm, along with the Deutsch–Jozsa algorithm and Simon's algorithm, demonstrated for the first time that a quantum computer could provide an **exponential advantage** over a classical computer for a non-trivial problem. This established the field of **quantum complexity theory** and proved that quantum computation is fundamentally more powerful than classical computation for certain tasks.

2. Threat to Cryptography (Cryptanalysis)

The algorithm's ability to efficiently break the **RSA cryptosystem**—which relies on the computational difficulty of factoring large numbers—is its most significant real-world implication.

- Shor's algorithm is the single greatest threat to current **public-key cryptography** and is the primary motivation for global investment in quantum computing and the development of **post-quantum cryptography** (new cryptographic methods that are secure against both classical and quantum attacks).

3. Driver of Quantum Hardware Development

The profound nature of the threat provided a clear, high-value goal for quantum engineers. The complexity of running Shor's algorithm—which requires a large number of stable, high-fidelity qubits—drives the development of quantum hardware capable of tackling large-scale quantum circuits.

Q15 Describe the working steps of Grover's Search Algorithm and explain the role of the oracle and diffusion operator.

Grover's Search Algorithm is a quantum algorithm that finds a unique target item x_0 in an unsorted database of N items with a **quadratic speedup** ($O(\sqrt{N})$) compared to the classical search complexity of $O(N)$.

The algorithm works by repeatedly applying a single operation—the **Grover Iteration**—to amplify the probability amplitude of the target state while simultaneously suppressing all non-target states.

Working Steps of the Algorithm

The algorithm uses n qubits, where $N = 2^n$ is the total size of the search space.

Step 1: Initialization and Superposition

The process begins by applying the **Hadamard gate (H)** to all n qubits, creating a uniform superposition of all possible states (all N items):

$$|\Psi_0\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

In this state, every item, including the marked target item $|x_0\rangle$, has the same amplitude, $\frac{1}{\sqrt{N}}$.

Step 2: The Grover Iteration

The core of the algorithm is the repeated application of the **Grover Operator (G)**, which is defined as:

$$\mathbf{G} = \mathbf{DO}$$

This operator is applied approximately $\mathbf{R} \approx \frac{\pi}{4} \sqrt{N}$ times. Each iteration increases the amplitude of the target state.

Step 3: Measurement

After the optimal number of iterations $\mathbf{R}$, a measurement is performed on the n qubits. Due to the amplification, the probability of measuring the state $|x_0\rangle$ is very close to 1.

Role of the Oracle and Diffusion Operator

The Grover Iteration consists of two fundamental components, each playing a distinct role in achieving amplitude amplification:

1. The Oracle (O) – The Marking Step

The quantum oracle, often denoted $\mathbf{U}_f$, serves as the **identifier** or **marker** for the target item x_0 . It is a black-box function that recognizes the solution but does not yet amplify it.

- **Working Principle:** The oracle acts on the superposition state by flipping the **phase** of the target state $|x_0\rangle$ while leaving all other states untouched.

$$|x\rangle \xrightarrow{\mathbf{O}} (-1)^{f(x)} |x\rangle$$

where $f(x) = 1$ only if $x = x_0$, and $f(x) = 0$ otherwise.

- **Role:** By marking the solution with a negative sign (a phase flip), the oracle creates a distinction between the solution and all other states, setting the stage for the next step.

2. The Diffusion Operator (D) – The Amplification Step

The diffusion operator, often called the **inversion about the average** operator, is a transformation that uses the phase flip from the oracle to amplify the marked state's amplitude.

- **Working Principle:** The operator effectively performs a reflection of the state vector about the initial average amplitude. Since the target state $|x_0\rangle$ has a negative phase, it now lies "below" the average. Reflecting about the average dramatically boosts the target's amplitude above the average, while simultaneously reducing the amplitudes of the unmarked states.
- **Role:** This operator is responsible for the **amplitude amplification**—the actual physical phenomenon that provides the speedup. Each application rotates the state vector closer to the target state $|x_0\rangle$.

Together, the oracle marks the target, and the diffusion operator uses that mark to selectively amplify the target's amplitude until it dominates the probability distribution.